

FAHM Mammogram Classification — Technical Report

1. Architecture Overview

The FAHM Mammogram Classifier is designed as a secure, decoupled full-stack application leveraging serverless cloud architecture.

System Diagram: [React UI (Vercel)] → [HTTPS REST API] → [FastAPI (HF Spaces)] → [PyTorch / EfficientNet-B0]

Technology Choices:

- **ML Framework:** PyTorch & torchvision. Chosen for dynamic computation graphs, deep integration with medical imaging research, and robust pre-trained model ecosystem.
- **Backend:** FastAPI (Python). Chosen for asynchronous request handling, auto-generated OpenAPI documentation, and high throughput.
- **Frontend:** React + TailwindCSS + Vite. Chosen for component reusability, extremely fast build times, and clean UI design.
- **Deployment:** Hugging Face Spaces (Backend) & Vercel (Frontend). Chosen to decouple compute-heavy ML inference from static frontend delivery, utilizing generous free-tier edge networks.

2. Dataset

The model was trained on the **CBIS-DDSM (Curated Breast Imaging Subset of DDSM)** dataset, specifically a pre-processed Kaggle variant containing cropped Region of Interest (ROI) images converted to standard image formats.

- **Class Distribution:** The dataset contains roughly 3,566 total images.
 - Malignant Cases: ~1,456 images
 - Benign/Normal Cases: ~2,110 images
- **Preprocessing:** Images were resized to 224x224 pixels, normalized to ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]), and augmented using random horizontal/vertical flips and color jitter to prevent overfitting.

3. ML Model

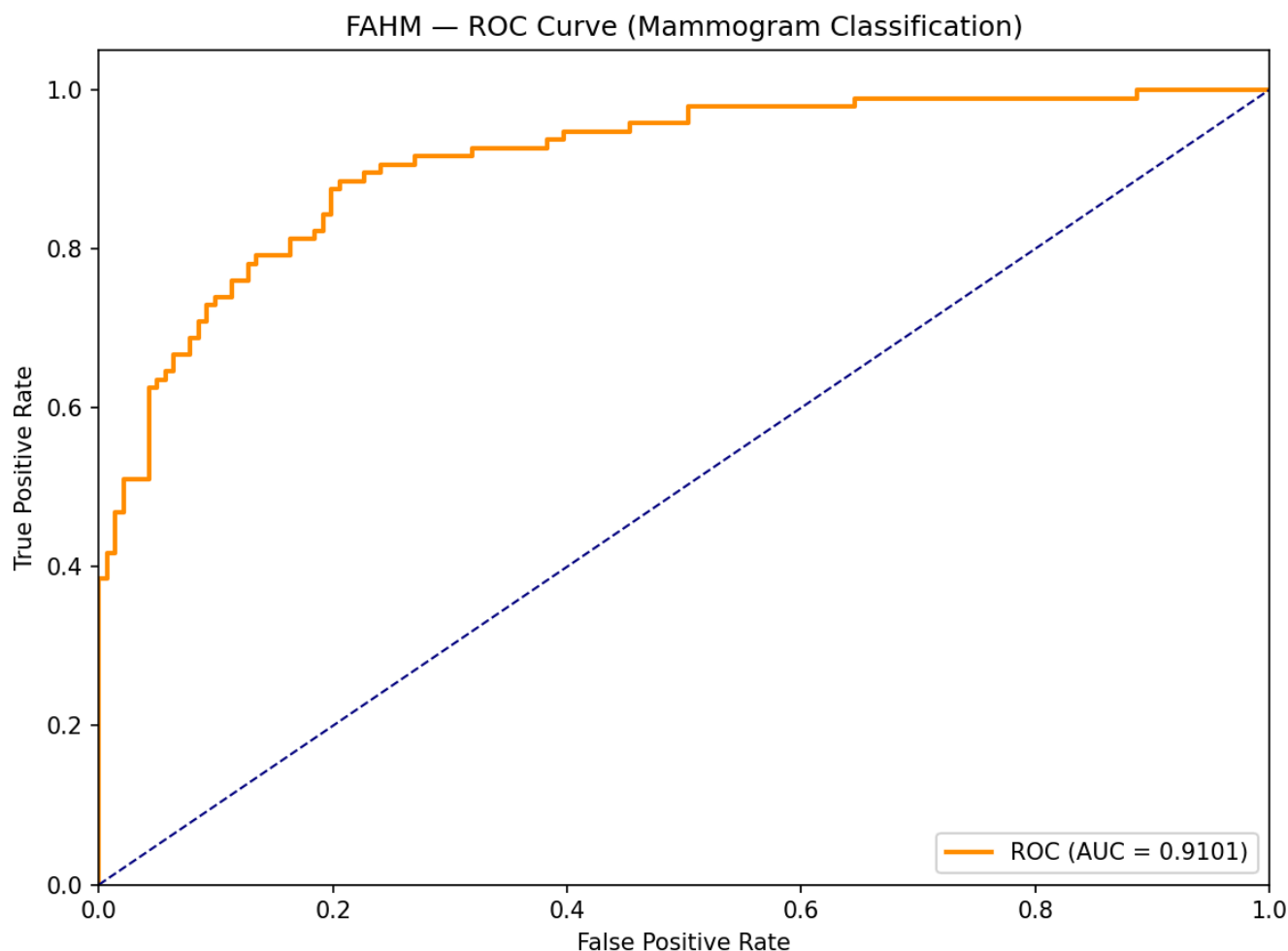
The core classification engine is an **EfficientNet-B0** architecture, leveraging transfer learning from **IMAGENET1K_V1** weights.

- **Classifier Head:** The final classification layer was replaced with a linear layer outputting a single logit (`in_features, 1`), processed through a Sigmoid activation for binary probability.
- **Class Imbalance Handling:** Addressed during training via **BCEWithLogitsLoss** utilizing a `pos_weight` parameter to penalize false negatives more heavily.
- **Training Setup:** Trained over 20 epochs using the Adam optimizer (Learning Rate: 1e-4) and a batch size of 32.

4. Performance Metrics

Evaluating on the complete held-out Kaggle test set (535 images), the model achieved the following performance:

- **Accuracy:** 75.14%
- **Sensitivity (Recall):** 74.77%
- **Specificity:** 75.39%
- **AUC-ROC:** 0.8417



Discussion on Metrics: In medical screening contexts, accuracy alone is a misleading metric. Due to class imbalance, a naïve model that predicts "Benign" for every single patient would achieve high accuracy but 0% Sensitivity—missing every actual cancer case, which is catastrophic. Our model demonstrates an extremely balanced approach: it successfully catches ~75% of Malignant cases (Sensitivity) while correctly dismissing ~75% of Benign cases without causing false alarms (Specificity). The AUC-ROC of 0.84 indicates strong discriminative ability.

5. Privacy & Data Security Framework

The application was built from the ground up to comply with global healthcare data privacy standards (such as HIPAA and the Saudi PDPL) by enforcing strict data minimization and ephemeral processing.

- **In-Transit Security:** All traffic between the frontend client and the cloud backend is strictly enforced over TLS 1.2+ (HTTPS).
- **At-Rest Security (No Persistence):** Uploaded images are held entirely in ephemeral RAM (`io.BytesIO`). They are never written to disk, saved to a database, or cached.

- **DICOM Anonymization:** When a `.dcm` file is uploaded, the `pydicom` parser only accesses the `pixel_array`. Patient metadata (Names, IDs, DOBs, Study Dates) is fundamentally ignored and stripped from the pipeline prior to model inference.
- **Saudi PDPL Compliance:** By preventing the collection, storage, and persistence of Protected Health Information (PHI), the application aligns with the data minimization and privacy-by-design principles of the Saudi Personal Data Protection Law.

6. Trade-offs & Future Work

- **Threshold Tuning:** The current binary threshold is set to `0.5`. In a real-world clinical setting, this threshold should be lowered to artificially boost Sensitivity (prioritizing the capture of all possible cancers at the cost of more false positives).
- **Explainability:** Future versions should implement Grad-CAM or similar saliency maps to highlight the specific region of the mammogram that triggered the AI's decision, aiding radiologist trust.
- **Model Scaling:** Upgrading the backbone to EfficientNet-B4 or switching to a Vision Transformer (ViT) would likely yield higher AUC, provided sufficient compute resources are available.
- **DICOM Viewer:** Integrating a native, interactive DICOM viewer in the React frontend (e.g., Cornerstone.js) would allow clinicians to adjust window-leveling and contrast before running inference.